Pruebas de SQL Injection

1. Introducción

En esta práctica se ha trabajado con Damn Vulnerable Web Application (DVWA) con el objetivo de comprender cómo se comporta una aplicación web vulnerable ante ataques del tipo SQL Injection, uno de los riesgos más críticos según OWASP Top 10.

2. Desarrollo del Caso Práctico

APARTADO 1: Creación del entorno de pruebas

Se ha preparado una máquina virtual con Ubuntu, donde se han instalado los servicios necesarios para ejecutar DVWA:

- apache2
- mysql-server
- php
- php-gd
- php-mysql

Después, se clonó el repositorio oficial en /var/www/html:
sudo git clone https://github.com/ethicalhack3r/DVWA

Una vez configurado Apache y MySQL, se accedió al aplicativo desde:
http://localhost/DVWA

```
campus@ubuntu-campus:~$ sudo apt update
Des:1 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Obj:2 http://es.archive.ubuntu.com/ubuntu noble InRelease
Des:3 http://es.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Des:4 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [1.309 kB
```

```
escargados 7.435 kB en 2s (3.599 kB/s)
eyendo lista de paquetes... Hecho
reando árbol de dependencias... Hecho
eyendo la información de estado... Hecho
e pueden actualizar 5 paquetes. Ejecute «apt list --upgradable» para verlos.
ampus@ubuntu-campus:~$ sudo apt install apache2 mysql-server php php-gd php-mysql -

eyendo lista de paquetes... Hecho
reando árbol de dependencias... Hecho
eyendo la información de estado... Hecho
pache2 ya está en su versión más reciente (2.4.58-1ubuntu8.8).
ysql-server ya está en su versión más reciente (8.0.43-0ubuntu0.24.04.2).
hp ya está en su versión más reciente (2:8.3+93ubuntu2).
hp-gd ya está en su versión más reciente (2:8.3+93ubuntu2).
hp-mysql ya está en su versión más reciente (2:8.3+93ubuntu2).
 actualizados, 0 nuevos se instalarán, 0 para eliminar y 5 no actualizados.
ampus@ubuntu-campus:~$
```

```
$_DVWA = array();
$_DVWA[ 'db_server' ]   = getenv('DB_SERVER') ?: '127.0.0.1';
$_DVWA[ 'db_database' ] = getenv('DB_DATABASE') ?: 'dvwa';
$_DVWA[ 'db_user' ]     = getenv('DB_USER') ?: 'dvwa_user';
$_DVWA[ 'db_password' ] = getenv('DB_PASSWORD') ?: 'password';
$_DVWA[ 'db_port']      = getenv('DB_PORT') ?: '3306';
```

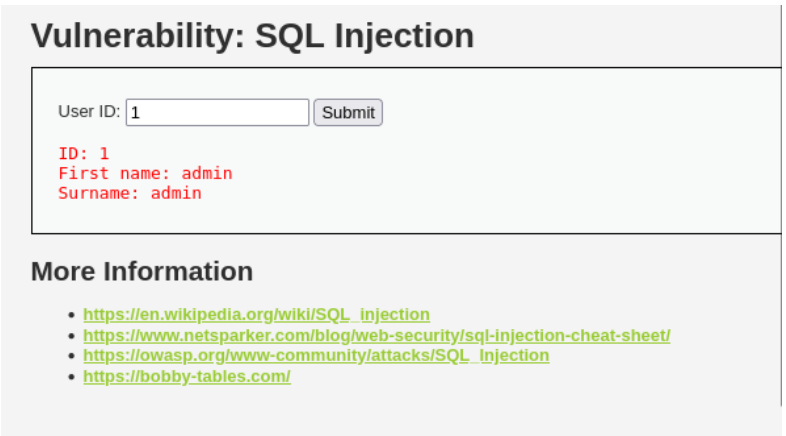
```
mysql> CREATE DATABASE dvwa;
ERROR 1007 (HY000): Can't create database 'dvwa'; database exists
mysql> CREATE USER 'dvwa_user'@'localhost' IDENTIFIED BY 'password';
ERROR 1396 (HY000): Operation CREATE USER failed for 'dvwa_user'@'localhost'
mysql> DELETE ALL FROM dvwa;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corr
esponds to your MySQL server version for the right syntax to use near 'ALL FROM dvwa
' at line 1
mysql> DROP DATABASE dvwa;
Query OK, 1 row affected (0,04 sec)

mysql> CREATE DATABASE dvwa;
Query OK, 1 row affected (0,01 sec)

mysql> CREATE USER 'dvwa_user'@'localhost' IDENTIFIED BY 'password';
ERROR 1396 (HY000): Operation CREATE USER failed for 'dvwa_user'@'localhost'
mysql> DROP USER 'dvwa_user'@'localhost';
Query OK, 0 rows affected (0,01 sec)

mysql> CREATE USER 'dvwa_user'@'localhost' IDENTIFIED BY 'password';
Query OK, 0 rows affected (0,01 sec)

mysql> GRANT ALL PRIVILEGES ON dvwa.* TO 'dvwa_user'@'localhost';
Query OK, 0 rows affected (0,01 sec)

mysql> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0,01 sec)

mysql> EXIT;
Bye
campus@ubuntu-campus:/var/www/html/DVWA/config$
```

APARTADO 2: Pruebas de vulnerabilidades – SQL Injection

1. Pruebas con IDs normales (1, 2, 3…)

La aplicación devuelve correctamente los registros asociados, demostrando que la consulta SQL está construida concatenando directamente la entrada del usuario.

La query sería:
SELECT first_name, last_name FROM users WHERE id = '1';



2. Pruebas con tres comillas simples (''')

El envío del payload provoca un error en DVWA (nivel Low).
Ocurre porque la comilla simple rompe la cadena SQL, generando una consulta inválida.

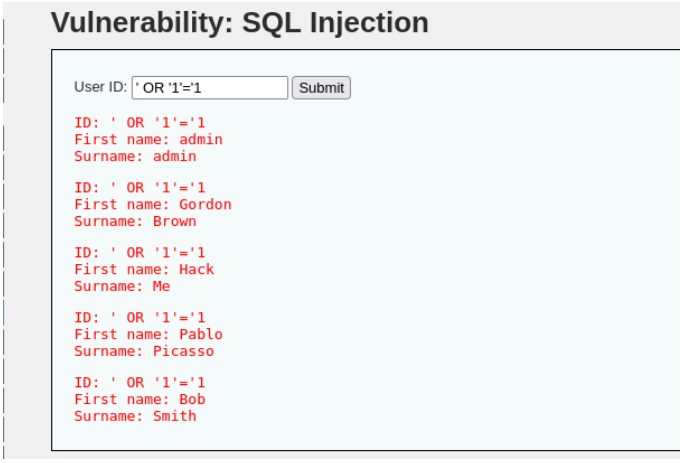
La query sería:
SELECT first_name, last_name FROM users WHERE id = '''';

3. SQL Injection con ' OR '1'='1

Este payload es un clásico ataque que fuerza la condición TRUE: ' OR '1'='1

La query sería:
SELECT first_name, last_name FROM users WHERE id = '' OR '1'='1';

El resultado es que la base de datos devuelve todos los usuarios, lo cual confirma la vulnerabilidad.



4. Cambio a nivel de seguridad "Impossible"

Cuando DVWA está en nivel Impossible, ya no es posible realizar inyecciones porque:

- usa prepared statements (consultas parametrizadas)
- validación y saneamiento de entrada
- funciones de escape seguras
- no permite concatenar directamente el input

Por ejemplo:
$stmt = $db->prepare('SELECT first_name, last_name FROM users WHERE id = ?');
$stmt->bind_param('i', $id);

En este caso, tanto "' como ' OR '1'='1 dejan de provocar errores o efectos inesperados.
(Incluye capturas donde se muestra que no se filtra ningún dato adicional.)

3. Preguntas Finales

1. ¿Cómo se ha comportado el aplicativo web cuando ha recibido 3 comillas simples (' ' ')?

El aplicativo devolvió un error SQL, indicando que la query estaba mal construida.
Ocurre porque las comillas rompen la estructura de la SQL.

---

2. ¿Qué query crees que ha ejecutado la base de datos?

La consulta fue algo equivalente a:
SELECT first_name, last_name FROM users WHERE id = '';

---

3. ¿Cómo se ha comportado el aplicativo web cuando ha recibido ' OR '1'='1? ¿Qué consulta ha realizado la base de datos?

La query ha devuelto todos los registros de la base de datos, ya que la condición '1'='1' es siempre verdadera.

La query sería:
SELECT first_name, last_name FROM users WHERE id = '' OR '1'='1';

---

4. ¿Qué diferencia ves en el código cuando está en modo Low y cuando está en modo Impossible? ¿Cómo crees que afecta a las vulnerabilidades?

Modo Low:

- Concatenación directa del input a la query
- No hay sanitización
- No se usan consultas preparadas
- El usuario puede alterar completamente la query

Ejemplo típico:
$id = $_GET['id'];
$query = "SELECT * FROM users WHERE id = '$id'";

SQL Injection Source

vulnerabilities/sqli/source/low.php

```php
<?php

if( isset( $_REQUEST[ 'Submit' ] ) ) {
    // Get input
    $id = $_REQUEST[ 'id' ];

    switch ($_DVWA['SQLI_DB']) {
        case MYSQL:
            // Check database
            $query  = "SELECT first_name, last_name FROM users WHERE user_id = '$id';";
            $result = mysqli_query($GLOBALS["___mysqli_ston"],  $query ) or die( '<pre>' . ((is_obje

            // Get results
            while( $row = mysqli_fetch_assoc( $result ) ) {
                // Get values
                $first = $row["first_name"];
                $last  = $row["last_name"];

                // Feedback for end user
                echo "<pre>ID: {$id}<br />First name: {$first}<br />Surname: {$last}</pre>";
            }

            mysqli_close($GLOBALS["___mysqli_ston"]);
            break;
        case SQLITE:
            global $sqlite_db_connection;

            #$sqlite_db_connection = new SQLite3($_DVWA['SQLITE_DB']);
            #$sqlite_db_connection->enableExceptions(true);

            $query  = "SELECT first_name, last_name FROM users WHERE user_id = '$id';";
            #print $query;

            try {
                $results = $sqlite_db_connection->query($query);
            } catch (Exception $e) {
                echo 'Caught exception: ' . $e->getMessage();
                exit();
            }

            if ($results) {
                while ($row = $results->fetchArray()) {
                    // Get values
                    $first = $row["first_name"];
                    $last  = $row["last_name"];

                    // Feedback for end user
                    echo "<pre>ID: {$id}<br />First name: {$first}<br />Surname: {$last}</pre>";
                }
            } else {
                echo "Error in fetch ".$sqlite_db->lastErrorMsg();
            }
            break;
    }
}

?>
```

Modo Impossible:

- Uso de prepared statements
- Validación estricta de datos
- No existe concatenación directa
- El parámetro del usuario no puede modificar la estructura de la query

Ejemplo:
$stmt = $db->prepare('SELECT * FROM users WHERE id = ?');
$stmt->bind_param('i', $id);

Impacto en la vulnerabilidad:

- En modo Low → totalmente vulnerable
- En modo Impossible → imposible realizar SQL Injection

SQL Injection Source

vulnerabilities/sqli/source/impossible.php

```php
<?php

if( isset( $_GET[ 'Submit' ] ) ) {
    // Check Anti-CSRF token
    checkToken( $_REQUEST[ 'user_token' ], $_SESSION[ 'session_token' ], 'index.php' );

    // Get input
    $id = $_GET[ 'id' ];

    // Was a number entered?
    if(is_numeric( $id )) {
        $id = intval ($id);
        switch ($_DVWA['SQLI_DB']) {
            case MYSQL:
                // Check the database
                $data = $db->prepare( 'SELECT first_name, last_name FROM users WHERE user_id = (:id)
                $data->bindParam( ':id', $id, PDO::PARAM_INT );
                $data->execute();
                $row = $data->fetch();

                // Make sure only 1 result is returned
                if( $data->rowCount() == 1 ) {
                    // Get values
                    $first = $row[ 'first_name' ];
                    $last  = $row[ 'last_name' ];

                    // Feedback for end user
                    echo "<pre>ID: {$id}<br />First name: {$first}<br />Surname: {$last}</pre>";
                }
                break;
            case SQLITE:
```

```
                global $sqlite_db_connection;

                $stmt = $sqlite_db_connection->prepare('SELECT first_name, last_name FROM users WHEF
                $stmt->bindValue(':id',$id,SQLITE3_INTEGER);
                $result = $stmt->execute();
                $result->finalize();
                if ($result !== false) {
                    // There is no way to get the number of rows returned
                    // This checks the number of columns (not rows) just
                    // as a precaution, but it won't stop someone dumping
                    // multiple rows and viewing them one at a time.

                    $num_columns = $result->numColumns();
                    if ($num_columns == 2) {
                        $row = $result->fetchArray();

                        // Get values
                        $first = $row[ 'first_name' ];
                        $last  = $row[ 'last_name' ];

                        // Feedback for end user
                        echo "<pre>ID: {$id}<br />First name: {$first}<br />Surname: {$last}</pre>";
                    }
                }

                break;
        }
    }
}

// Generate Anti-CSRF token
generateSessionToken();

?>
```

```
campus@campus-pps:~$ sudo systemctl status apache2
[sudo] password for campus:
● apache2.service - The Apache HTTP Server
     Loaded: loaded (/usr/lib/systemd/system/apache2.service; enable>
     Active: active (running) since Mon 2025-11-17 17:48:26 UTC; 14m>
       Docs: https://httpd.apache.org/docs/2.4/
    Process: 10557 ExecStart=/usr/sbin/apachectl start (code=exited,>
   Main PID: 10561 (apache2)
      Tasks: 8 (limit: 4589)
     Memory: 18.9M (peak: 19.6M)
        CPU: 363ms
     CGroup: /system.slice/apache2.service
             ├─10561 /usr/sbin/apache2 -k start
             ├─10564 /usr/sbin/apache2 -k start
             ├─10565 /usr/sbin/apache2 -k start
             ├─10566 /usr/sbin/apache2 -k start
             ├─10567 /usr/sbin/apache2 -k start
             ├─10568 /usr/sbin/apache2 -k start
             ├─10594 /usr/sbin/apache2 -k start
```

```
MariaDB service is already enabled.
MariaDB service is already running.

Default credentials:
Username: root

Password:[No password just hit Enter]
Enter SQL user:
Enter SQL password (press Enter for no password):
SQL commands executed successfully.
Configuring DVWA...
Configuring permissions...
Configuring PHP...
Apache service is already enabled.
Restarting Apache...
DVWA has been installed successfully. Access http://localhost/DVWA to
 get started.
Credentials:
Username: admin
Password: password

With ❤ by IamCarron
```